МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Государственный Университет Управления

Кафедра: Институт Информационных Систем

Лабораторная работа № 6

Реализация линейных структур данных и анализ трудоёмкости их операций

Дисциплина: Теория алгоритмов

Направление подготовки: 38.03.05 «Бизнес-информатика»

Язык реализации: Python 3.10+

Вариант: 14

Выполнил: студент группы БИ 2-1 Лапшенков Григорий Витальевич

Проверил: [должность, ФИО преподавателя]

[Город], 2026

1. Цель работы и задание варианта

1.1. Цель работы

Изучить линейные структуры данных (стек, очередь, связный список), реализовать их на основе массива и на основе связанных узлов и экспериментально установить, как выбор структуры данных определяет трудоёмкость типовых операций.

1.2. Задание варианта 14

Таблица 1. Исходные данные варианта

Параметр	Значение
Структуры для реализации	стек, очередь
Исследуемая операция	реализация очереди **через два стека**
Размеры n	5000; 10 000; 20 000
Прикладная задача	моделирование потока заявок
Дополнительные требования	T1 — проверка на малом примере; T2 — время для собственной реализации и для встроенного `list`; T3 — график $T(n)$ и определение класса трудоёмкости; T4 — расход памяти на 10 000 элементов (`sys.getsizeof`)

Реализованы: односвязный список (для оценки узловой реализации), стек на динамическом массиве и стек на связанных узлах, очередь на двух стеках, а также для сравнения — очередь на кольцевом буфере и очередь на `list` с извлечением `pop(0)`. Исследуется поведение очереди на двух стеках: её амортизированная и худшая трудоёмкость и сравнение с альтернативами.

2. Описание реализованных структур и операций

2.1. Стеки

Стек — АТД с дисциплиной LIFO и операциями `push`, `pop`, `top`. Реализации:

- **`ArrayStack`** — на динамическом массиве (`list`); `push` — `append`, $O(1)$ амортизированно, `pop` — $\Theta(1)$;
- **`LinkedStack`** — на односвязном списке `LinkedList`: вершина — голова списка, `push` и `pop` — вставка и удаление в начале за $\Theta(1)$.

Оба стека ведут счётчик `ops` числа операций `push` и `pop`.

2.2. Односвязный список

Класс `LinkedList` со ссылкой на хвост поддерживает `push_front`, `push_back`, `insert`, `pop_front`, `get`, `find`, `__len__`. Вставка и удаление в начале — $\Theta(1)$, доступ по индексу и поиск — $\Theta(n)$.

2.3. Очередь на двух стеках

Очередь (FIFO) строится на двух стеках: **inbox** принимает новые элементы, **outbox** выдаёт элементы. При `enqueue(x)` выполняется `inbox.push(x)`. При `dequeue()`: если `outbox` пуст, все элементы последовательно перекладываются из `inbox` в `outbox` (`outbox.push(inbox.pop())`), что **меняет порядок на обратный**, так что самый старый элемент оказывается на вершине `outbox`; затем выполняется `outbox.pop()`.

Трудоёмкость. Одна операция `dequeue`, вызвавшая перекладывание, стоит $\Theta(k)$, где k — число элементов в `inbox`. Однако перекладывание выполняется только при пустом `outbox`, поэтому каждый элемент за свою жизнь проходит четыре стековые операции: `push` в `inbox`, `pop` из `inbox`, `push` в `outbox`, `pop` из `outbox`. Тогда любые n операций стоят не более $4n$ стековых операций, и **амортизированная стоимость** каждой операции — $O(1)$. Этот факт проверяется экспериментально счётчиком: на элемент приходится ровно 4,00 стековых операции.

Худший случай одной операции остаётся $\Theta(n)$ — это принципиальная особенность амортизированных оценок.

Неудачный вариант (`NaiveTwoStackQueue`): при каждом `dequeue` все элементы переносятся в другой стек и обратно. Одна операция стоит $\Theta(n)$ всегда, серия из n извлечений — $\Theta(n^2)$.

2.4. Прочие очереди (для сравнения)

- **Кольцевой буфер** (`CircularQueue`): индексы головы и счётчик, $tail = (head + count) \bmod m$; обе операции $\Theta(1)$ в худшем случае, но ёмкость фиксирована.
- **`list` с `append` / `pop(0)`** (`ListQueue`): извлечение из начала сдвигает все элементы, $\Theta(n)$ на операцию.

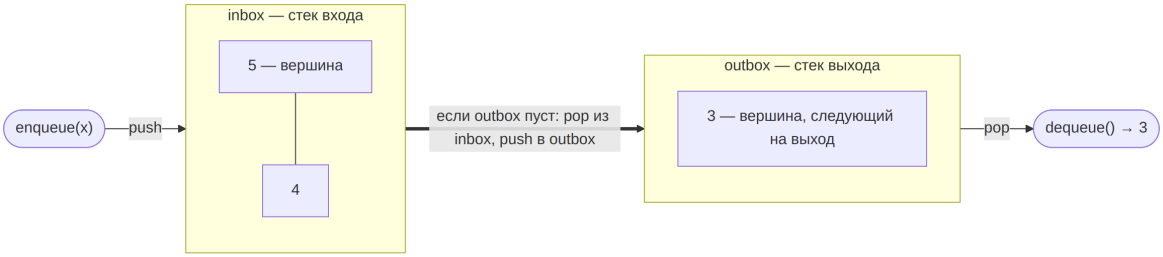
2.5. Ожидаемая трудоёмкость

Операция	Два стека	Наивный перенос	Кольцевой буфер	`list`
enqueue	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$O(1)$ амортиз.
dequeue, одна операция	$\Theta(n)$ в худшем случае	$\Theta(n)$ всегда	$\Theta(1)$	$\Theta(n)$
dequeue, амортизированно	$O(1)$	$\Theta(n)$	$\Theta(1)$	$\Theta(n)$
n извлечений	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n)$	$\Theta(n^2)$

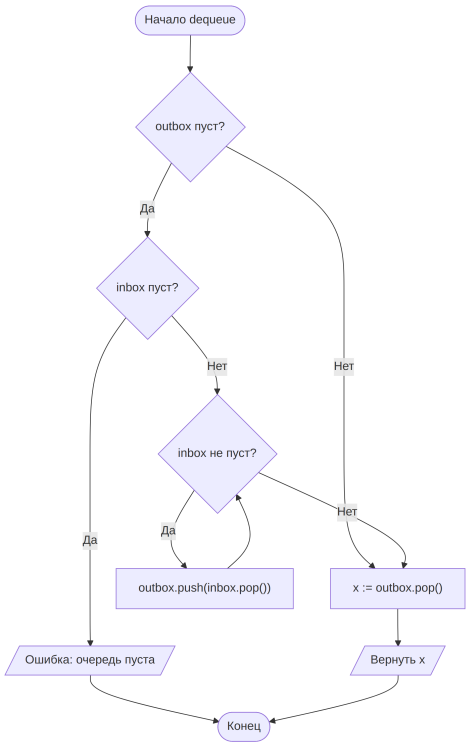
3. Схемы структур данных

3.1. Схема очереди на двух стеках

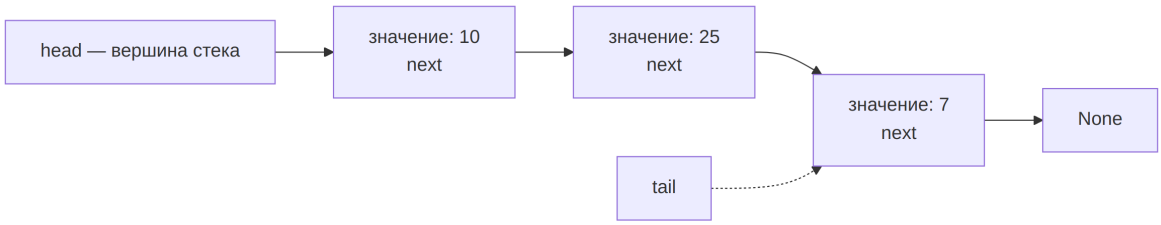
Состояние: в очередь поставлены элементы 3, 4, 5; элемент 3 уже переложен в `outbox`.



3.2. Блок-схема операции dequeue



3.3. Схема односвязного списка (стек на узлах)



4. Листинг программы

Программа содержит все структуры, самопроверку (сверка очередей с `collections.deque` на случайной последовательности операций), демонстрацию на малом примере, замеры, оценку памяти, моделирование потока заявок и построение графиков.

```
"""
Лабораторная работа № 6. Вариант 14.
Стек и очередь. Исследуемая операция: реализация очереди через два стека.
n = 5000, 10 000, 20 000. Прикладная задача: моделирование потока заявок.
"""
import gc
import json
import math
import random
import statistics
import sys
import time
from collections import deque

SIZES = [5_000, 10_000, 20_000]
REPEATS = 5
```

```python
# ------------------------------------------------------ односвязный список
class Node:
    """Узел односвязного списка."""
    __slots__ = ("value", "next")

    def __init__(self, value, nxt=None):
        self.value = value
        self.next = nxt


class LinkedList:
    """Односвязный список со ссылкой на хвост."""

    def __init__(self):
        self.head = None
        self.tail = None
        self.size = 0

    def push_front(self, value):
        self.head = Node(value, self.head)
        if self.tail is None:
            self.tail = self.head
        self.size += 1

    def push_back(self, value):
        node = Node(value)
        if self.tail is None:
            self.head = self.tail = node
        else:
            self.tail.next = node
            self.tail = node
        self.size += 1

    def insert(self, index, value):
        """Вставка перед элементом с номером index — O(index)."""
        if not 0 <= index <= self.size:
            raise IndexError("Индекс вне диапазона")
        if index == 0:
            return self.push_front(value)
        if index == self.size:
            return self.push_back(value)
        prev = self.head
        for _ in range(index - 1):
            prev = prev.next
        prev.next = Node(value, prev.next)
        self.size += 1

    def pop_front(self):
        if self.head is None:
            raise IndexError("Список пуст")
        value = self.head.value
        self.head = self.head.next
        if self.head is None:
            self.tail = None
        self.size -= 1
        return value

    def get(self, index):
        if not 0 <= index < self.size:
            raise IndexError("Индекс вне диапазона")
```

```
        node = self.head
        for _ in range(index):
            node = node.next
        return node.value

    def find(self, value):
        """Индекс первого вхождения значения или -1 — θ(n)."""
        node, i = self.head, 0
        while node:
            if node.value == value:
                return i
            node, i = node.next, i + 1
        return -1

    def to_list(self):
        result, node = [], self.head
        while node:
            result.append(node.value)
            node = node.next
        return result

    def __len__(self):
        return self.size


# ---------------------------------------------------------------- стеки
class ArrayStack:
    """Стек на динамическом массиве (list). Счётчик ops — число операций push/pop."""

    def __init__(self):
        self.data = []
        self.ops = 0

    def push(self, value):
        self.ops += 1
        self.data.append(value)

    def pop(self):
        if not self.data:
            raise IndexError("Стек пуст")
        self.ops += 1
        return self.data.pop()

    def top(self):
        if not self.data:
            raise IndexError("Стек пуст")
        return self.data[-1]

    def __len__(self):
        return len(self.data)

    def __bool__(self):
        return bool(self.data)


class LinkedStack:
    """Стек на односвязном списке: вершина — голова списка."""

    def __init__(self):
        self.lst = LinkedList()
```

```
        self.ops = 0

    def push(self, value):
        self.ops += 1
        self.lst.push_front(value)

    def pop(self):
        self.ops += 1
        return self.lst.pop_front()

    def top(self):
        return self.lst.get(0)

    def __len__(self):
        return len(self.lst)

    def __bool__(self):
        return len(self.lst) > 0


# ------------------------------------------------------------- очереди
class TwoStackQueue:
    """Очередь на двух стеках: вход (inbox) и выход (outbox).
    Элементы перекладываются из inbox в outbox только когда outbox пуст,
    поэтому каждый элемент перекладывается не более одного раза."""

    def __init__(self, stack_cls=ArrayStack):
        self.inbox, self.outbox = stack_cls(), stack_cls()

    def enqueue(self, value):
        self.inbox.push(value)                      # O(1)

    def _transfer(self):
        while self.inbox:
            self.outbox.push(self.inbox.pop())

    def dequeue(self):
        if not self.outbox:
            if not self.inbox:
                raise IndexError("Очередь пуста")
            self._transfer()                        # O(n) в худшем случае
        return self.outbox.pop()

    def front(self):
        if not self.outbox:
            self._transfer()
        return self.outbox.top()

    def stack_ops(self):
        return self.inbox.ops + self.outbox.ops

    def __len__(self):
        return len(self.inbox) + len(self.outbox)


class NaiveTwoStackQueue:
    """Неудачный вариант: при каждом извлечении все элементы переносятся
    во второй стек и обратно — O(n) на каждую операцию dequeue."""

    def __init__(self, stack_cls=ArrayStack):
```

```
        self.main, self.aux = stack_cls(), stack_cls()

    def enqueue(self, value):
        self.main.push(value)

    def dequeue(self):
        if not self.main:
            raise IndexError("Очередь пуста")
        while self.main:
            self.aux.push(self.main.pop())
        value = self.aux.pop()
        while self.aux:
            self.main.push(self.aux.pop())
        return value

    def stack_ops(self):
        return self.main.ops + self.aux.ops

    def __len__(self):
        return len(self.main)


class CircularQueue:
    """Очередь на кольцевом буфере фиксированной ёмкости."""

    def __init__(self, capacity):
        self.data = [None] * capacity
        self.capacity = capacity
        self.head = 0
        self.count = 0

    def enqueue(self, value):
        if self.count == self.capacity:
            raise OverflowError("Очередь переполнена")
        self.data[(self.head + self.count) % self.capacity] = value
        self.count += 1

    def dequeue(self):
        if self.count == 0:
            raise IndexError("Очередь пуста")
        value = self.data[self.head]
        self.data[self.head] = None
        self.head = (self.head + 1) % self.capacity
        self.count -= 1
        return value

    def __len__(self):
        return self.count


class ListQueue:
    """Очередь на встроенном list: извлечение из начала — pop(0), O(n)."""

    def __init__(self):
        self.data = []

    def enqueue(self, value):
        self.data.append(value)

    def dequeue(self):
```

```
        return self.data.pop(0)

    def __len__(self):
        return len(self.data)


# ---------------------------------------------- T1: проверка на малом примере
def demo():
    """Работа очереди на двух стеках на малом примере (вывод после каждой операции)."""
    q = TwoStackQueue()
    script = [("enq", 1), ("enq", 2), ("enq", 3), ("deq", None), ("enq", 4),
              ("deq", None), ("deq", None), ("enq", 5), ("deq", None),
              ("deq", None), ("deq", None)]
    lines = []
    for op, v in script:
        if op == "enq":
            q.enqueue(v)
            title = f"enqueue({v})"
            result = ""
        else:
            title = "dequeue()"
            try:
                result = f" -> {q.dequeue()}"
            except IndexError:
                result = " -> ошибка: очередь пуста"
        lines.append(f"{title + result:<28} inbox={q.inbox.data}  outbox={q.outbox.data}")
    return lines


def self_test():
    """Сверка всех очередей с эталоном deque на случайной последовательности операций."""
    rng = random.Random(3)
    for make in (TwoStackQueue, lambda: TwoStackQueue(LinkedStack),
                 NaiveTwoStackQueue, lambda: CircularQueue(1000), ListQueue):
        q, ref = make(), deque()
        for i in range(5000):
            if ref and rng.random() < 0.45:
                assert q.dequeue() == ref.popleft()
            elif len(ref) < 900:
                q.enqueue(i)
                ref.append(i)
        assert len(q) == len(ref)
    print("Самопроверка пройдена: порядок FIFO совпадает с collections.deque.")


# ------------------------------------------------------------------ замеры
def best_median(action, repeats=REPEATS):
    times = []
    for _ in range(repeats):
        gc.collect()
        t0 = time.perf_counter()
        action()
        times.append(time.perf_counter() - t0)
    return statistics.median(times)


def filled(make, n):
    q = make()
    for i in range(n):
        q.enqueue(i)
```

```
    return q


def measure_queues(n):
    """Время n постановок и n извлечений для каждой реализации, с."""
    makers = {"два стека": TwoStackQueue,
              "кольцевой буфер": lambda: CircularQueue(n),
              "list (append / pop(0))": ListQueue}
    out = {}
    for name, make in makers.items():
        def enq(make=make):
            q = make()
            for i in range(n):
                q.enqueue(i)

        def deq(make=make):
            q = filled(make, n)
            t0 = time.perf_counter()
            for _ in range(n):
                q.dequeue()
            return time.perf_counter() - t0

        t_enq = best_median(enq)
        t_deq = statistics.median(deq() for _ in range(REPEATS))
        out[name] = {"enqueue": t_enq, "dequeue": t_deq}
    return out


def measure_naive(n, pairs=100):
    """Время одной пары dequeue+enqueue при длине очереди n (после первого переноса), с."""
    def run(make):
        q = filled(make, n)
        q.dequeue()                  # прогрев: первое извлечение выполняет перенос
        q.enqueue(-1)
        t0 = time.perf_counter()
        for i in range(pairs):
            q.dequeue()
            q.enqueue(i)
        return (time.perf_counter() - t0) / pairs
    return {"наивный перенос": statistics.median(run(NaiveTwoStackQueue) for _ in range(3)),
            "два стека": statistics.median(run(TwoStackQueue) for _ in range(3))}


def dequeue_spike(n):
    """Время каждого из n извлечений из очереди на двух стеках, с."""
    q = filled(TwoStackQueue, n)
    times = []
    for _ in range(n):
        t0 = time.perf_counter()
        q.dequeue()
        times.append(time.perf_counter() - t0)
    return times


def count_stack_ops(n):
    """Стековых операций на элемент после n постановок и n извлечений."""
    q = filled(TwoStackQueue, n)
    for _ in range(n):
        q.dequeue()
    return q.stack_ops() / n
```

```python
def memory_report(n=10_000):
    """T4: оценка памяти контейнеров на n элементах (sys.getsizeof, байт)."""
    arr = []
    for i in range(n):
        arr.append(i)
    exact = list(range(n))
    ll = LinkedList()
    for i in range(n):
        ll.push_back(i)
    nodes, node = 0, ll.head
    while node:
        nodes += sys.getsizeof(node)
        node = node.next

    class DictNode:                          # узел без __slots__ (для сравнения)
        def __init__(self, value, nxt=None):
            self.value, self.next = value, nxt
    dn = DictNode(0)
    dict_node = sys.getsizeof(dn) + sys.getsizeof(dn.__dict__)
    q = filled(TwoStackQueue, n)
    q.dequeue()                              # перекладываем всё во второй стек
    return {
        "n": n,
        "list_append": sys.getsizeof(arr),
        "list_exact": sys.getsizeof(exact),
        "linked": sys.getsizeof(ll) + nodes,
        "node": sys.getsizeof(ll.head),
        "node_dict": dict_node,
        "two_stack": sum(sys.getsizeof(s.data) for s in (q.inbox, q.outbox)),
        "deque": sys.getsizeof(deque(range(n))),
        "int_obj": sys.getsizeof(n),
    }


# ------------------------------------------- прикладная задача: поток заявок
def simulate(num_requests, lam, mu, seed, queue_factory=TwoStackQueue):
    """Система массового обслуживания M/M/1: один обработчик, очередь FIFO.
    Заявки поступают с интенсивностью lam, обслуживаются с интенсивностью mu.
    Возвращает среднее время ожидания в очереди, среднюю длину очереди,
    максимальную длину очереди и признак соблюдения FIFO."""
    rng = random.Random(seed)
    queue = queue_factory()
    t, server_free_at, next_id, served_id = 0.0, 0.0, 0, -1
    arrival_time = {}
    arrivals, waits, area, max_len, last_t, fifo_ok = [], [], 0.0, 0, 0.0, True
    for _ in range(num_requests):
        t += rng.expovariate(lam)
        arrivals.append(t)
    i = 0
    # дискретно-событийное моделирование
    while i < num_requests or len(queue):
        next_arrival = arrivals[i] if i < num_requests else math.inf
        if len(queue) and server_free_at <= next_arrival:
            now = max(server_free_at, last_t)
            area += len(queue) * (now - last_t)
            last_t = now
            req = queue.dequeue()
            fifo_ok &= (req > served_id)
```

```
            served_id = req
            waits.append(now - arrival_time.pop(req))
            server_free_at = now + rng.expovariate(mu)
        else:
            area += len(queue) * (next_arrival - last_t)
            last_t = next_arrival
            queue.enqueue(next_id)
            arrival_time[next_id] = next_arrival
            next_id += 1
            i += 1
            max_len = max(max_len, len(queue))
            if server_free_at < next_arrival and len(queue) == 1:
                server_free_at = next_arrival
    return {"wait": statistics.mean(waits), "avg_len": area / last_t,
            "max_len": max_len, "fifo": fifo_ok, "served": len(waits), "waits": waits}


# ---------------------------------------------------------------- графики
def make_plots(res):
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt
    ns = [r["n"] for r in res["queues"]]

    fig, axes = plt.subplots(1, 2, figsize=(11, 4.3))
    for ax, op, title in ((axes[0], "enqueue", "n постановок в очередь"),
                          (axes[1], "dequeue", "n извлечений из очереди")):
        for name in res["queues"][0]["t"]:
            ax.plot(ns, [r["t"][name][op] * 1e3 for r in res["queues"]], "o-", label=name)
        ax.set_xscale("log"); ax.set_yscale("log")
        ax.set_xlabel("n"); ax.set_ylabel("время, мс"); ax.set_title(title)
        ax.grid(True, which="both", alpha=.3); ax.legend(fontsize=8)
    plt.tight_layout(); plt.savefig("img/lr6_time.png", dpi=170); plt.close()

    sp = res["spike"]
    plt.figure(figsize=(6.8, 4.2))
    plt.semilogy(range(1, len(sp) + 1), [t * 1e6 for t in sp], ".", ms=2)
    plt.xlabel("номер операции dequeue"); plt.ylabel("время одной операции, мкс")
    plt.title(f"Очередь на двух стеках: время каждого извлечения (n = {len(sp):,})".replace(",", "
↪   "))
    plt.grid(True, which="both", alpha=.3)
    plt.tight_layout(); plt.savefig("img/lr6_spike.png", dpi=170); plt.close()

    plt.figure(figsize=(6.8, 4.2))
    for name in ("наивный перенос", "два стека"):
        plt.plot(ns, [r["naive"][name] * 1e6 for r in res["queues"]], "o-", label=name)
    plt.yscale("log"); plt.xscale("log")
    plt.xlabel("длина очереди n"); plt.ylabel("время пары dequeue + enqueue, мкс")
    plt.title("Наивный перенос и перенос по требованию")
    plt.grid(True, which="both", alpha=.3); plt.legend()
    plt.tight_layout(); plt.savefig("img/lr6_naive.png", dpi=170); plt.close()

    sim = res["sim_trace"]
    fig, axes = plt.subplots(1, 2, figsize=(11, 3.8))
    axes[0].plot(sim["len_t"], sim["len_v"], lw=.6)
    axes[0].set_xlabel("номер события"); axes[0].set_ylabel("длина очереди")
    axes[0].set_title("Длина очереди заявок (ρ = 0,8)")
    axes[1].hist(sim["waits"], bins=50, color="#55a868")
    axes[1].set_xlabel("время ожидания в очереди"); axes[1].set_ylabel("число заявок")
    axes[1].set_title("Распределение времени ожидания")
```

```
    for ax in axes:
        ax.grid(True, alpha=.3)
    plt.tight_layout(); plt.savefig("img/lr6_sim.png", dpi=170); plt.close()


# ---------------------------------------------------------------------- main
if __name__ == "__main__":
    self_test()
    res = {"demo": demo()}
    print("\n".join(res["demo"]))

    res["queues"] = []
    for n in SIZES:
        t = measure_queues(n)
        naive = measure_naive(n)
        res["queues"].append({"n": n, "t": t, "naive": naive, "ops_per_item": count_stack_ops(n)})
        print(f"n = {n}: " + "; ".join(f"{k}: {v['enqueue']:.5f}/{v['dequeue']:.5f}" for k, v in
         ↪  t.items()),
              "| naive", {k: round(v * 1e6, 1) for k, v in naive.items()}, flush=True)

    spike = dequeue_spike(SIZES[-1])
    res["spike"] = spike
    print(f"извлечение: первое {spike[0] * 1e6:.1f} мкс, медиана остальных "
          f"{statistics.median(spike[1:]) * 1e6:.2f} мкс, среднее {statistics.mean(spike) *
           ↪  1e6:.2f} мкс")

    res["memory"] = memory_report(10_000)
    print(res["memory"])

    # прикладная задача: M/M/1, lam = 0.8, mu = 1.0 (ρ = 0.8)
    lam, mu = 0.8, 1.0
    rho = lam / mu
    res["sim_theory"] = {"lam": lam, "mu": mu, "rho": rho, "wq": rho / (mu - lam),
                         "lq": rho ** 2 / (1 - rho)}
    res["sim"] = []
    for n in SIZES:
        reps = [simulate(n, lam, mu, seed) for seed in range(30)]
        res["sim"].append({"n": n,
                           "wait_mean": statistics.mean(r["wait"] for r in reps),
                           "wait_sd": statistics.stdev(r["wait"] for r in reps),
                           "len_mean": statistics.mean(r["avg_len"] for r in reps),
                           "len_sd": statistics.stdev(r["avg_len"] for r in reps),
                           "max_len": statistics.mean(r["max_len"] for r in reps),
                           "fifo": all(r["fifo"] for r in reps),
                           "served": min(r["served"] for r in reps)})
        print(res["sim"][-1])
    # идентичность результатов для разных реализаций очереди при одинаковых случайных числах

    class DequeQueue:
        def __init__(self):
            self.d = deque()

        def enqueue(self, v):
            self.d.append(v)

        def dequeue(self):
            return self.d.popleft()

        def __len__(self):
            return len(self.d)
```

```
    a = simulate(5000, lam, mu, 0)
    b = simulate(5000, lam, mu, 0, DequeQueue)
    res["sim_same"] = (a["wait"] == b["wait"] and a["avg_len"] == b["avg_len"])
    print("совпадение с deque:", res["sim_same"])

    # трасса одного прогона для графиков
    lens = []

    class TraceQueue(TwoStackQueue):
        def enqueue(self, v):
            super().enqueue(v)
            lens.append(len(self))

        def dequeue(self):
            r = super().dequeue()
            lens.append(len(self))
            return r
    trace = simulate(5000, lam, mu, 1, TraceQueue)
    res["sim_trace"] = {"len_t": list(range(len(lens))), "len_v": lens, "waits": trace["waits"]}
    with open("res/lr6.json", "w", encoding="utf-8") as f:
        json.dump(res, f, ensure_ascii=False)
    make_plots(res)
```

5. Таблицы измерений и графики

5.1. Условия измерений

Параметр	Значение
Операционная система	Linux 6.18 (x86_64), виртуальная машина
Процессор	Intel Xeon, 2,10 ГГц, 1 виртуальное ядро
Оперативная память	4 ГБ
Интерпретатор	CPython 3.12.3
Способ замера времени	`time.perf_counter()`, перед каждым прогоном вызывается `gc.collect()`

Время — медиана 5 повторов (для сравнения наивного и обычного переноса — медиана 3 повторов по 100 пар операций). Для серии «n извлечений» очередь предварительно заполняется (заполнение в замер не входит).

5.2. Проверка корректности на малом примере (T1)

Работа очереди на двух стеках с выводом содержимого стеков после каждой операции:

```
enqueue(1)                  inbox=[1]  outbox=[]
enqueue(2)                  inbox=[1, 2]  outbox=[]
enqueue(3)                  inbox=[1, 2, 3]  outbox=[]
dequeue() -> 1              inbox=[]  outbox=[3, 2]
enqueue(4)                  inbox=[4]  outbox=[3, 2]
dequeue() -> 2              inbox=[4]  outbox=[3]
dequeue() -> 3              inbox=[4]  outbox=[]
enqueue(5)                  inbox=[4, 5]  outbox=[]
dequeue() -> 4              inbox=[]  outbox=[5]
dequeue() -> 5              inbox=[]  outbox=[]
dequeue() -> ошибка: очередь пуста inbox=[]  outbox=[]
```

Элементы выходят в порядке поступления (1, 2, 3, 4, 5): очередь соблюдает FIFO; видно, что перекладывание происходит только при пустом `outbox`: при первом извлечении (из `inbox = [1, 2, 3]` получается `outbox = [3, 2]` после выдачи элемента 1) и при извлечении элемента 4 (`outbox` пуст, `inbox = [4, 5]`); остальные извлечения берут элемент прямо из

`outbox`. Извлечение из пустой очереди приводит к ошибке. Автоматическая самопроверка (5000 случайных операций для пяти реализаций, сверка с `collections.deque`) пройдена.

5.3. Время операций (T2)

Таблица 2. Время n постановок и n извлечений, мс

n	Два стека: enqueue	Два стека: dequeue	Кольц. буфер: enqueue	Кольц. буфер: dequeue	`list`: append	`list`: pop(0)
5 000	0,316	1,102	0,418	0,488	0,165	0,681
10 000	0,655	2,196	0,872	0,991	0,313	5,269
20 000	1,430	4,717	1,869	2,062	0,685	27,755

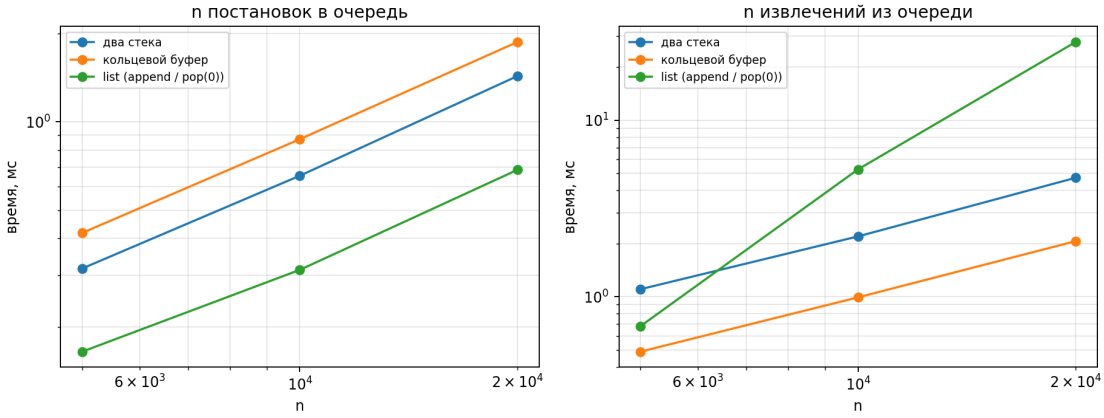


Рис. 1. Время n постановок и n извлечений для разных реализаций

Постановка в очередь на двух стеках (1,43 мс при $n = 20\,000$) медленнее, чем `list.append` (0,69 мс), так как каждая постановка проходит два уровня вызовов (`enqueue` → `push`) и увеличивает счётчик `ops`, тогда как `ListQueue.enqueue` выполняет один вызов `append`. Серия из n извлечений из `list` с `pop(0)` при $n = 20\,000$ занимает 27,8 мс против 4,7 мс у очереди на двух стеках и 2,1 мс у кольцевого буфера: сдвиг массива при каждом `pop(0)` делает серию квадратичной.

5.4. Определение класса трудоёмкости (T3)

Таблица 3. Отношение времён при удвоении n и вывод о классе трудоёмкости

Операция (n штук)	Структура	Теория для серии	$T(2n)/T(n)$: 5000→10 000 и 10 000→20 000	Вывод
enqueue	два стека	$\Theta(n)$	2,1 и 2,2	линейный рост
dequeue	два стека	$\Theta(n)$ амортизированно	2,0 и 2,1	линейный рост
enqueue	кольцевой буфер	$\Theta(n)$	2,1 и 2,1	линейный рост
dequeue	кольцевой буфер	$\Theta(n)$	2,0 и 2,1	линейный рост
append	`list`	$\Theta(n)$ амортизированно	1,9 и 2,2	линейный рост
pop(0)	`list`	$\Theta(n^2)$	7,7 и 5,3	рост быстрее линейного (квадратичный)
пара dequeue + enqueue	два стека, наивный перенос	$\Theta(n)$ на пару	2,0 и 1,9	линейный рост времени одной операции

При линейной зависимости $T(2n)/T(n) \approx 2$, при квадратичной — ≈ 4. Все серии постановок и извлечений для очереди на двух стеках, кольцевого буфера и `list.append` дают значения около 2 — линейный рост серии, то есть $\Theta(1)$ на операцию. Серия `pop(0)` растёт быстрее линейной: отношения 7,7 и 5,3 превышают теоретические 4,

поскольку помимо квадратичной составляющей сказывается рост размера сдвигаемого массива относительно кэша процессора (вероятная причина; в данных этого опыта она не проверялась). Отношение существенно превышает 2, что исключает линейную зависимость серии `pop(0)`; класс $\Theta(n^2)$ согласуется с этим результатом и с теорией (сдвиг массива при каждой операции).

Для наивного переноса время **одной пары** `dequeue + enqueue` растёт пропорционально длине очереди (отношение ≈ 2 при удвоении n) — одна операция $\Theta(n)$. Для переноса по требованию эта величина постоянна:

Таблица 4. Время одной пары `dequeue + enqueue` при длине очереди n (после первого перекладывания)

Длина очереди n	Наивный перенос, мкс на пару	Перенос по требованию, мкс на пару	Отношение
5 000	1 260,4	0,17	7 599
10 000	2 475,9	0,17	14 913
20 000	4 791,4	0,17	27 733

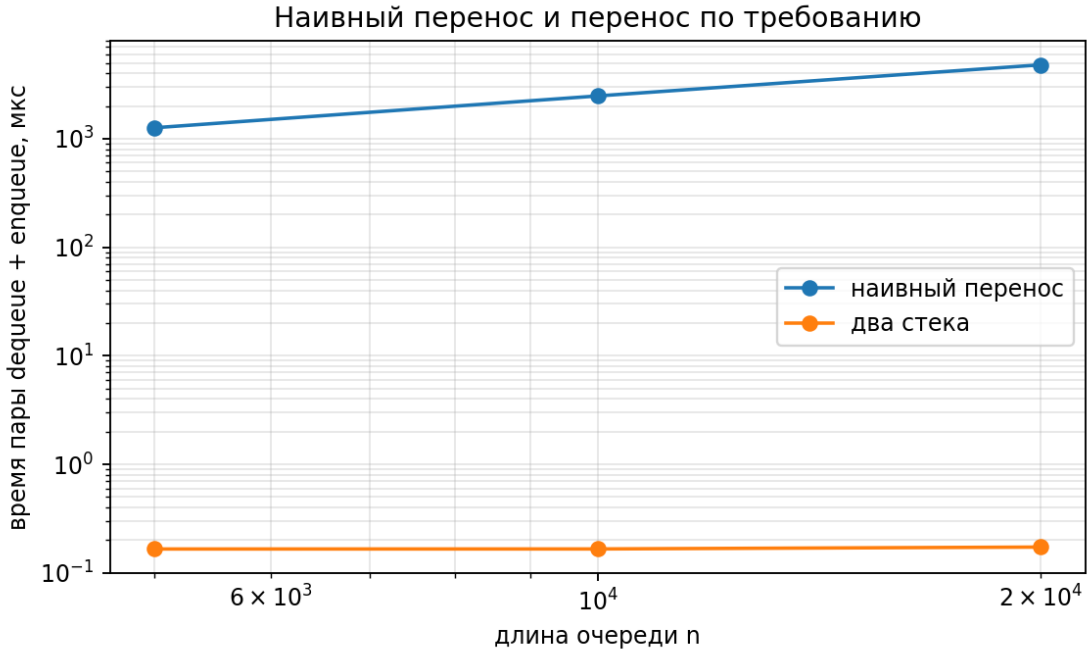


Рис. 2. Наивный перенос и перенос по требованию

Амортизация и худший случай. Время каждого из $n = 20\,000$ извлечений из заполненной очереди на двух стеках показано на рис. 3. Первое извлечение, вызвавшее перекладывание всех элементов, заняло 2 401 мкс; медиана остальных — 0,15 мкс, среднее по всем — 0,27 мкс. Первая операция в 16 556 раз дольше типичной; максимум среди остальных — 19,7 мкс. Амортизированная стоимость мала (среднее по серии лишь вдвое выше медианы), но один «выброс» $\Theta(n)$ реален.

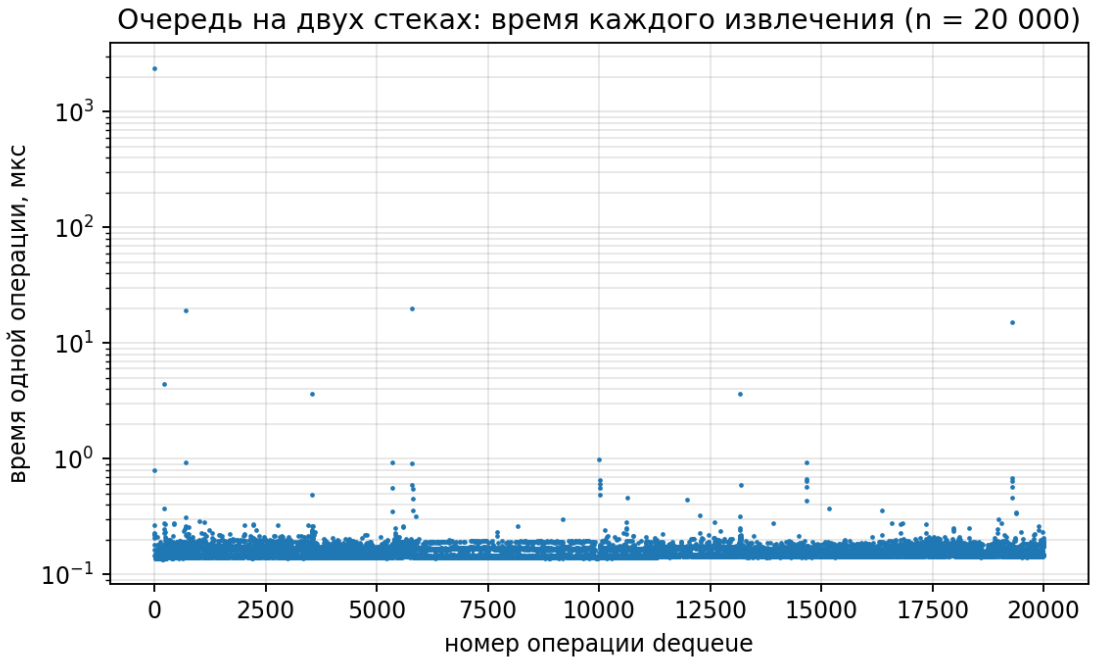


Рис. 3. Время каждого извлечения (очередь на двух стеках, n = 20 000)

5.5. Расход памяти (T4)

Размеры контейнеров на $n = 10\,000$ элементов, измеренные `sys.getsizeof` (размер самих значений в таблице не учтён: объект `int` занимает около 28 байт и одинаков во всех структурах — порядка 280 000 байт на 10 000 значений).

Таблица 5. Память контейнеров на 10 000 элементов

Структура ($n = 10\,000$)	Размер контейнера, байт	Байт на элемент
`list`, заполнен через `append` (стек на массиве)	85 176	8,52
`list(range(n))` (точная ёмкость)	80 056	8,01
Очередь на двух стеках (два `list`)	85 232	8,52
`collections.deque`	83 128	8,31
Односвязный список (узлы с `__slots__`)	480 048	48,00

Массив хранит только 8-байтовые указатели; заполнение через `append` даёт небольшой запас ёмкости (около 6 % сверх точного размера). Узел односвязного списка — отдельный объект: даже с `__slots__` он занимает 48 байт (заголовок объекта, две ссылки `value` и `next` и служебный заголовок сборщика мусора), то есть связный список расходует память в 5,6 раза больше массива. Без `__slots__` узел обходится примерно в 344 байт (объект и словарь атрибутов, по сумме `getsizeof`). Очередь на двух стеках занимает на 56 байт больше одного массива (объект второго, пустого списка), то есть по памяти не хуже массива. Помимо объёма, узлы рассеяны по памяти, что ухудшает использование кэша процессора.

6. Сводная таблица «операция — структура — теоретическая оценка — эксперимент»

Операция	Структура	Теория	Эксперимент
enqueue (серия из n)	очередь на двух стеках	$\Theta(1)$ на операцию	$T(2n)/T(n) \approx 2{,}1$ и 2,2
dequeue (серия из n)	очередь на двух стеках	$O(1)$ амортизированно; одна операция $\Theta(n)$	$T(2n)/T(n) \approx 2{,}0$ и 2,1; первая операция 2 401 мкс при медиане 0,15 мкс
стековых операций на элемент	очередь на двух стеках	4 (push + pop + pop + push)	4,00

Операция	Структура	Теория	Эксперимент
dequeue + enqueue при длине n	наивный перенос через два стека	$\Theta(n)$ на пару	1 260; 2 476; 4 791 мкс (рост ×2 при ×2 по n)
enqueue / dequeue (серия из n)	кольцевой буфер	$\Theta(1)$ на операцию	$T(2n)/T(n) \approx 2{,}0$ и 2,1
append (серия из n)	`list`	$O(1)$ амортизированно	$T(2n)/T(n) \approx 1{,}9$ и 2,2
pop(0) (серия из n)	`list`	$\Theta(n)$ на операцию	$T(2n)/T(n) \approx 7{,}7$ и 5,3
push / pop, top	стек на массиве (`list`)	$\Theta(1)$	входит в измерения очереди на двух стеках
память на элемент	массив / связный список	ссылка / узел со ссылкой	8,5 / 48,0 байт

7. Прикладная задача: моделирование потока заявок

7.1. Постановка и модель

Моделируется простейшая система массового обслуживания M/M/1: заявки поступают с интенсивностью $\lambda = 0{,}8$ (экспоненциальные интервалы), один обработчик обслуживает заявки с интенсивностью $\mu = 1{,}0$ (экспоненциальное время обслуживания), ожидающие заявки образуют очередь FIFO. Нагрузка $\rho = \lambda/\mu = 0{,}8$. В качестве очереди использована очередь на двух стеках. Для установившегося режима теория даёт

$$W_q = \frac{\rho}{\mu - \lambda} = 4{,}0, \qquad L_q = \lambda W_q = \frac{\rho^2}{1-\rho} = 3{,}2,$$

где W_q — среднее время ожидания в очереди, L_q — средняя длина очереди (без заявки на обслуживании). Для каждого N выполнено 30 независимых прогонов, в таблице — среднее и стандартное отклонение по прогонам.

Таблица 6. Результаты моделирования ($\rho = 0{,}8$)

Число заявок N	Среднее ожидание в очереди	Теория W_q	Средняя длина очереди	Теория L_q	Макс. длина очереди (среднее)	FIFO
5 000	3,87 ± 0,71	4,00	3,09 ± 0,60	3,20	26,3	соблюдён
10 000	4,04 ± 0,41	4,00	3,23 ± 0,34	3,20	27,6	соблюдён
20 000	3,93 ± 0,25	4,00	3,15 ± 0,21	3,20	29,9	соблюдён

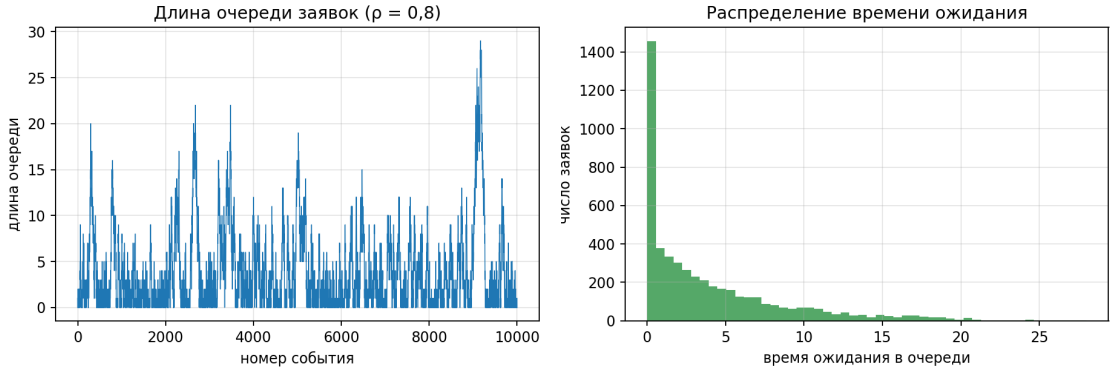


Рис. 4. Длина очереди заявок и распределение времени ожидания (один прогон, N = 5000)

Средние значения согласуются с теорией: время ожидания отличается от 4,0 менее чем на 0,13 (в пределах 1,5 стандартной ошибки средней по 30 прогонам), средняя длина очереди — от 3,2 не более чем на 0,11. Разброс между прогонами убывает с ростом N. Занижение среднего ожидания на 3 % при $N = 5000$ согласуется с влиянием старта с пустой очереди, но при такой погрешности статистически незначимо. Порядок обслуживания FIFO соблюдён во всех прогонах (номера обслуживаемых заявок возрастают), а результаты моделирования при одинаковых случайных числах **совпадают** с результатами при

использовании `collections.deque` в качестве очереди, что подтверждает корректность реализации.

7.2. Обоснование выбора структуры

Задача требует дисциплины FIFO и неограниченного заранее числа заявок.

- **Очередь на двух стеках** (требование варианта) корректна и имеет амортизированную стоимость $O(1)$: на элемент приходится четыре стековые операции, а серия из 20 000 извлечений занимает 4,7 мс. Недостаток — одна операция, выполняющая перекладывание, стоит $\Theta(n)$: в опыте 2 401 мкс против 0,15 мкс у типичного извлечения. В моделировании длина очереди невелика (в среднем максимум порядка 30 заявок), поэтому «выбросы» несущественны.
- **Кольцевой буфер** даёт $\Theta(1)$ в худшем случае без выбросов (2,1 мс на 20 000 извлечений), но имеет фиксированную ёмкость: для потока неизвестной длины нужно либо задавать запас, либо реализовывать расширение, при котором снова возникает одиночная операция $\Theta(n)$.
- **`list` с `pop(0)`** непригоден: серия из 20 000 извлечений медленнее кольцевого буфера примерно в 13 раз.
- В практическом коде на Python для такой задачи используется `collections.deque` ($\Theta(1)$ на операцию без выбросов); очередь на двух стеках оправдана, когда доступны только стековые операции, либо когда важна средняя пропускная способность, а разовая задержка допустима. Если же заявки должны обслуживаться с гарантированной задержкой на каждую операцию, следует выбирать кольцевой буфер или `deque`.

8. Выводы

1. Очередь на двух стеках корректна и обладает амортизированной трудоёмкостью $O(1)$: на каждый элемент приходится ровно четыре стековые операции, а время серии из n постановок или извлечений растёт линейно (отношение времён при удвоении n около 2). Каждый элемент перекладывается не более одного раза, поскольку перекладывание выполняется только при пустом `outbox`.
2. Амортизированная оценка не исключает дорогих отдельных операций: перекладывание $n = 20\,000$ элементов заняло 2 401 мкс — в 16 556 раз больше типичного извлечения (0,15 мкс). Для систем с жёсткими требованиями ко времени отклика это существенно.
3. Наивный перенос на каждое извлечение даёт $\Theta(n)$ на операцию: при длине очереди 20 000 одна пара операций занимает 4 791 мкс против 0,17 мкс при переносе по требованию (разница в 27 733 раз).
4. Очередь на `list` с `pop(0)` имеет квадратичную трудоёмкость серии извлечений; кольцевой буфер и очередь на двух стеках — линейную. Различие между $\Theta(1)$ и $\Theta(n)$ на операцию становится заметным уже при $n = 5000$–$20\,000$.
5. Связный список выигрывает у массива по асимптотике вставки и удаления в начале, но проигрывает по памяти (48 байт на узел против 8,5 байт на элемент массива в опыте — в 5,6 раза) и по доступу по индексу; поэтому структуру следует выбирать по тому, какие операции преобладают.
6. Моделирование потока заявок (M/M/1, $\rho = 0{,}8$) на очереди из двух стеков воспроизводит теоретические $W_q = 4{,}0$ и $L_q = 3{,}2$ в пределах статистической погрешности и совпадает с результатами при использовании `deque`, что подтверждает корректность реализации.

9. Ответы на контрольные вопросы

1. Чем абстрактный тип данных отличается от его реализации? Приведите пример АТД с двумя разными реализациями. АТД задаёт набор операций и их смысл (что делает структура), реализация определяет способ хранения и стоимость операций (как). Пример — АТД «очередь» с операциями `enqueue`, `dequeue`: в работе она реализована на кольцевом буфере, на двух стеках и на `list` с `pop(0)`. Интерфейс один и тот же, а трудоёмкость разная: при $n = 20\,000$ серия извлечений занимает 2,1 мс (буфер), 4,7 мс (два стека) и 27,8 мс (`pop(0)`).

2. Сформулируйте дисциплины LIFO и FIFO. Приведите по два прикладных примера. LIFO (стек) — последним пришёл, первым вышел; FIFO (очередь) — первым

пришёл, первым вышел. Примеры LIFO: откат действий (undo) в редакторе; стек вызовов функций и проверка скобок. Примеры FIFO: очередь печати; поток заявок в системе обслуживания (моделировался в пункте 7) и обход графа в ширину.

3. Почему доступ по индексу в массиве выполняется за $\Theta(1)$, а в связном списке за $\Theta(n)$? Массив занимает непрерывную область памяти, адрес элемента вычисляется как base $+ i \cdot$ size за постоянное время. Узлы списка разбросаны по памяти и связаны ссылками, поэтому, чтобы дойти до i-го узла, нужно последовательно пройти i ссылок — $\Theta(i)$, в худшем случае $\Theta(n)$ (метод `get` в нашей реализации).

4. Что такое амортизированная сложность? Почему вставка в конец динамического массива стоит $O(1)$ амортизированно? Амортизированная сложность — средняя стоимость операции по любой последовательности операций (а не по случайным данным): редкие дорогие операции оплачиваются многими дешёвыми. При заполнении динамический массив удваивает ёмкость и копирует n элементов — $\Theta(n)$, но копирования происходят при размерах $1, 2, 4, \ldots$, и суммарные затраты на n вставок $< 2n$, то есть $O(1)$ на вставку. Аналогично очередь на двух стеках: на элемент приходится ровно 4 стековые операции (измерено: 4,00).

5. Зачем в связном списке хранят ссылку на хвост? Чтобы добавление в конец (`push_back`, `enqueue`) выполнялось за $\Theta(1)$: без ссылки на хвост пришлось бы пройти весь список ($\Theta(n)$). В нашей реализации `LinkedList` хвост обновляется при каждой вставке. В односвязном списке хвост не ускоряет удаление последнего элемента (нужен предыдущий узел).

6. В чём преимущество двусвязного списка перед односвязным и какова его цена? Преимущества: обход в обе стороны; удаление заданного узла и удаление хвоста за $\Theta(1)$ (есть ссылка на предыдущий узел); удобство реализации дека. Цена: ещё одна ссылка в каждом узле (больше памяти — для 10 000 элементов узел односвязного списка уже занимает 48 байт) и более сложное обновление ссылок при вставке и удалении (четыре ссылки вместо двух), больше шансов нарушить инварианты.

7. Как устроен кольцевой буфер? Какую роль играет операция взятия остатка? Это массив фиксированной ёмкости m с индексом головы и счётчиком элементов; запись ведётся в позицию $(head + count) \bmod m$, чтение — из позиции $head$, после чего $head = (head + 1) \bmod m$. Остаток «заворачивает» индекс с конца массива в начало, поэтому освободившиеся при извлечении ячейки переиспользуются, а сдвигать элементы не нужно. Обе операции $\Theta(1)$ в худшем случае, цена — фиксированная ёмкость (при переполнении `enqueue` выдаёт ошибку).

8. Почему на практике массив часто быстрее связного списка, хотя асимптотика говорит об обратном? Асимптотика не учитывает константы и устройство памяти. Элементы массива лежат подряд, процессор эффективно использует кэш и предвыборку; узлы списка разбросаны по памяти, обход требует перехода по ссылкам (промахи кэша), и каждый узел — отдельное выделение памяти. В нашем опыте узел занимает 48 байт против 8,5 байт на элемент массива (в 5,6 раза больше, табл. 5). Поэтому выигрыш связного списка на вставке в начало проявляется лишь при больших объёмах и частых вставках/удалениях.

9. Как реализовать очередь с помощью двух стеков? Какова амортизированная стоимость операций? Входной стек `inbox` принимает `enqueue`; при `dequeue`, если выходной стек `outbox` пуст, все элементы переносятся из `inbox` в `outbox` (порядок разворачивается), затем выполняется `outbox.pop()`. Амортизированная стоимость `enqueue` и `dequeue` — $O(1)$: каждый элемент проходит не более четырёх стековых операций. Худшая одиночная операция — $\Theta(n)$: в опыте первое извлечение при $n = 20\,000$ заняло 2 401 мкс при типичных 0,15 мкс.

10. Какую структуру данных вы выбрали бы для журнала последних 100 событий и почему? Кольцевой буфер ёмкости 100 (в Python — `collections.deque(maxlen=100)`): добавление нового события и вытеснение самого старого за $\Theta(1)$, фиксированный объём памяти, нет сдвигов и дополнительных выделений, события читаются в порядке поступления. Связный список избыточен по памяти (48 байт на узел против 8 байт на ссылку в массиве), `list` с `pop(0)` стоит $\Theta(n)$ на вытеснение.